

Yenepoya (Deemed To Be University)

Project – High Level Design

on

Big Data Analytics on Energy Demand

Student Name: Aafreen John Nedungadan

Register Number: 23BCDSBA01

Industry Mentor:
Mr. Ujjwal Kumar

Guided by: Mr. Prashanth

Table of Contents

- 1. Introduction.**
 - 1.1. Scope of the document.
 - 1.2. Intended Audience.
 - 1.3. System overview.
- 2. System Design.**
 - 2.1. Application Design.
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design.
 - 2.5. Key Design Considerations.
 - 2.6. API Catalogue.
- 3. Data Design.**
 - 3.1. Data Model.
 - 3.2. Data Access Mechanism.
 - 3.3. Data Retention Policies.
 - 3.4. Data Migration.
- 4. Interfaces.**
- 5. State and Session Management.**
- 6. Caching.**
- 7. Non-Functional Requirements.**
 - 7.1. Security Aspects.
 - 7.2. Performance Aspects.
- 8. References.**

1. Introduction

1.1. Scope of the document: This High Level Design (HLD) document outlines the overall architecture and conceptual design of the “Big Data Analytics for Energy Demand” project. It presents the system’s high-level flow, major components, data model, database structure, and the conceptual approach used to demonstrate Big Data concepts. The scope is limited to the ETL pipeline developed in Jupyter Notebook using Python, Pandas, and SQLite. It covers loading the raw CSV file (energy_consumption_levels.csv with 8784 rows), cleaning the data, performing aggregations that simulate MapReduce, storing the results in a SQLite database, running SQL queries for reports, and creating visualizations. The document focuses on the “what” and “why” of the design rather than low-level implementation details.

1.2. Intended Audience: This HLD is primarily intended for the project guide, course instructor, and academic examiners who will evaluate the BCA final year project. It is also useful for fellow students and future developers who may refer to this work for learning ETL pipelines and Big Data concepts. The document is written in clear technical language so that anyone with basic knowledge of data analytics can understand the overall system design and its objectives.

1.3. System overview: The system is a self-contained local analytics pipeline that runs entirely inside a single Jupyter Notebook. It starts with loading the raw energy consumption data from a CSV file, cleans and transforms the data by renaming columns and creating a proper datetime field, performs monthly and hourly aggregations using Pandas GroupBy and resample operations, stores both the cleaned raw data and the aggregated summaries in a SQLite database, and finally

generates business reports through SQL queries and three professional charts using Seaborn and Matplotlib. The entire process is designed to be simple, reproducible, and educational. It demonstrates how Big Data concepts such as ETL and MapReduce can be applied effectively on a manageable dataset of 8784 rows without needing large-scale distributed systems or complex infrastructure.

2. System Design

2.1. Overall System Architecture: The system is designed as a self-contained, local analytics pipeline that runs entirely inside a single Jupyter Notebook. It follows a clear, linear structure that starts with the raw CSV file and ends with reports and visualizations. The architecture is divided into four logical layers: data ingestion, data transformation, persistent storage, and presentation layer. All operations are performed in-memory using Pandas, making the system fast and easy to run on any standard laptop. The design deliberately avoids complex distributed systems so that the focus remains on understanding core Big Data concepts like ETL and MapReduce simulation on a manageable dataset of 8784 rows.

2.2. Process Flow: The process flow is linear and follows the classic ETL pattern. It begins with loading the raw CSV file into a Pandas DataFrame. In the transformation phase, the data is cleaned by renaming the main column, creating a single datetime field from separate time components, dropping unnecessary columns, sorting the records chronologically, and removing any duplicate timestamps. Next, the aggregation phase creates two summary tables using Pandas GroupBy and resample operations. The monthly summary calculates total, average, and peak demand per month, while the hourly summary shows average and peak

demand for each hour of the day. Finally, all processed data is written to the SQLite database for persistent storage and future querying.

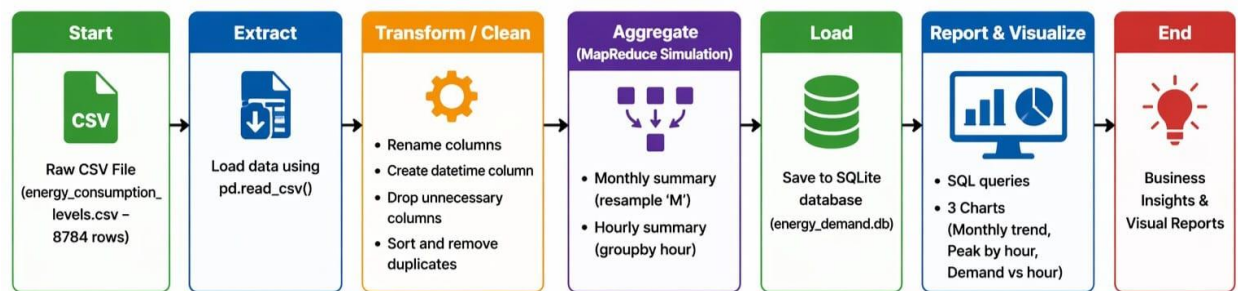


Figure 2.1: ETL Process Flow – Big Data Analytics for Energy Demand

2.3. Information Flow: Information flows from the raw CSV file to the Pandas DataFrame, then to cleaned data, aggregated tables, and finally to the SQLite database. SQL queries and charts are generated from the same processed data.

2.4. Components Design: The system consists of six major components: Dataset, Data Loading, Data Transformation, Aggregation (MapReduce Simulation), Storage & Reporting, and Visualization. Each component has a clear role and works together to complete the ETL pipeline.

2.5. Key Design Considerations: The design focuses on simplicity, educational value, and reproducibility. Pandas was chosen for its GroupBy and resample capabilities that simulate MapReduce. SQLite was selected for its lightweight nature and single-file storage. Clear print statements were added for transparency.

2.6. API Catalogue: No external APIs are used in the current version as the project works with a static CSV file. A real-time energy API can be added in future enhancements.

3. Data Design

The data model of this project is based on time-series energy consumption records. The core entity is hourly energy demand represented by the demand_mwh column (measured in Megawatt-hour). A single datetime column serves as the primary time dimension for all analysis. The model clearly separates the cleaned raw data from the aggregated summary tables. This separation allows both detailed granular records and high-level summarized insights to be available for reporting and visualization. The design is simple yet scalable, making it ideal for demonstrating Big Data concepts on a manageable dataset of 8784 rows. This relational structure also supports easy future expansion, such as adding temperature-based analysis or multi-region data, without redesigning the entire model.

Data Model – Big Data Analytics for Energy Demand

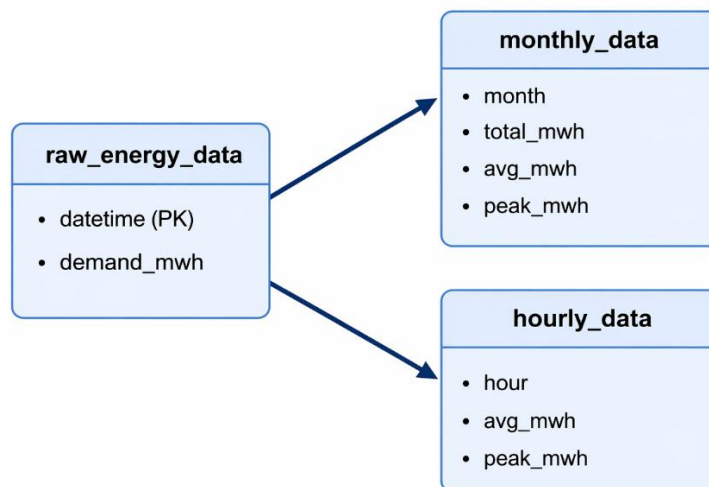


Figure 3.1: Data Model for Big Data Analytics on Energy Demand

3.2. Data Access Mechanism: All data access is handled uniformly through the Pandas library for simplicity and consistency. The raw input is read from the CSV file using `pd.read_csv()`. Processed DataFrames are written into the SQLite database using the `to_sql()` method with the `if_exists='replace'` option. For reporting and analysis, data is retrieved from the database using `pd.read_sql()`. This Pandas-centric approach eliminates the need for complex SQL scripting during normal operations and keeps the entire pipeline easy to maintain and understand. It also ensures high portability, allowing the same code to work across different environments with minimal changes.

3.3. Data Retention Policies: The original raw CSV file (`energy_consumption_levels.csv`) is kept completely unchanged and treated as a read-only source. The SQLite database file (`energy_demand.db`) is fully replaced on every run of the notebook to ensure that the latest cleaned and aggregated data is always available. Basic data quality checks such as row count and missing value verification are performed and displayed at every stage. This policy guarantees data freshness while preventing accumulation of outdated versions.

3.4. Data Migration: No data migration is required in the current version of the project because it works with a single static CSV file as the only source. All data is directly loaded into Pandas, cleaned, aggregated, and then written to the SQLite database in one seamless flow. In future enhancements, if the project moves to real-time data sources or cloud storage, a proper migration strategy can be added to transfer historical data without any loss or duplication. This approach keeps the current implementation lightweight and focused on core ETL functionality.

4. Interfaces

The system provides two primary interfaces that enable smooth interaction between the user, the data, and the final outputs.

The **Input Interface** is the raw CSV file named `energy_consumption_levels.csv`. This file acts as the single source of truth for the entire ETL pipeline. The Jupyter Notebook reads this file directly using the Pandas library, ensuring that no manual data entry is required.

The **Output Interface** consists of SQL-based reports and professional visualizations. After processing, the cleaned and aggregated data stored in the SQLite database can be queried using simple SQL statements through Pandas. Additionally, three clear charts are generated using Seaborn and Matplotlib.

The **Main User Interface** is the Jupyter Notebook itself. The notebook is organized into clearly numbered sections with descriptive markdown headings and informative print statements.

Overall, the interfaces are deliberately kept simple, self-contained, and user-friendly to match the academic nature of the project while still providing complete end-to-end functionality from raw data to actionable business insights. This design ensures that the evaluator can easily run the notebook and understand the results without any additional tools.

5. State and Session Management

The system is a single-run Jupyter Notebook pipeline and does not involve any multi-user web application or long-running sessions. Therefore, traditional user session management (such as login sessions, cookies, or authentication tokens) is not required in this project.

Data state is managed entirely through Pandas DataFrames during execution and is persisted permanently in the SQLite database. The pipeline maintains four clear and well-defined internal states that transition sequentially in a linear flow:

- **Raw State:** The data is loaded directly from the CSV file into a Pandas DataFrame.
- **Cleaned State:** After renaming columns, creating the datetime field, dropping unnecessary columns, sorting, and removing duplicates.
- **Aggregated State:** After performing monthly and hourly aggregations using GroupBy and resample operations (simulating MapReduce).
- **Stored State:** The final cleaned and aggregated data is saved into the SQLite database file (energy_demand.db) using the to_sql() method.

This state transition ensures that data moves smoothly from raw input to final output without any loss of information. The notebook itself acts as the temporary execution context, while the SQLite database serves as the persistent storage layer that holds the final state of the processed data across multiple runs. The clear separation of states also makes it very easy to debug any issues at any stage of the pipeline.

The design keeps state management simple, transparent, and easy to debug, which is ideal for an academic project. No complex state machines or session handling logic was needed because the pipeline runs from start to finish in one execution. This approach also improves reproducibility, allowing the examiner to clearly understand how data moves through different stages during evaluation.

State Transition in the ETL Pipeline

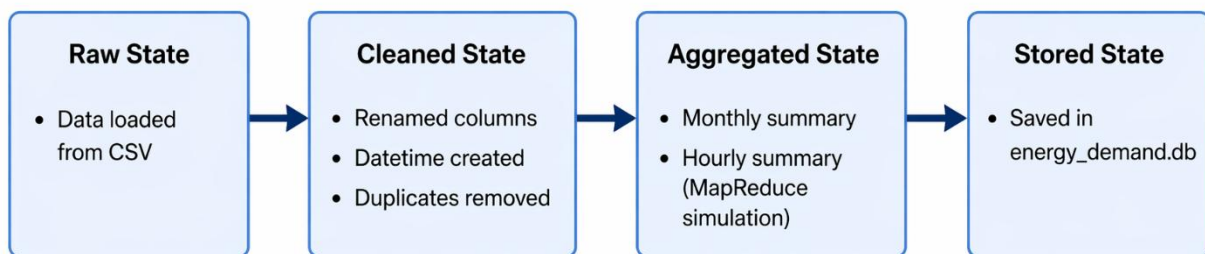


Figure 5.1: State Transition in the ETL Pipeline

This clear state transition approach significantly improves the transparency of the entire ETL process. It allows any evaluator to quickly trace how the raw data evolves into meaningful insights. Such clarity is particularly valuable during project demonstration.

6. Caching

The project is a simple, single-execution Jupyter Notebook pipeline with a relatively small dataset of 8784 rows. Therefore, advanced caching mechanisms such as Redis, Memcached, or application-level query caching are not implemented. All data processing is performed efficiently in-memory using Pandas DataFrames, which are highly optimized for this scale and provide excellent performance without the need for additional caching layers.

Pandas itself internally handles temporary data structures efficiently during operations like GroupBy, resample, and repeated DataFrame access. Once the data is cleaned and aggregated, it is immediately saved to the SQLite database file (energy_demand.db) using the `to_sql()` method. This database then acts as the primary persistent storage, allowing quick retrieval of processed data for SQL queries and visualizations without re-running the entire ETL pipeline.

This minimal caching approach keeps the project lightweight, easy to understand, and fully suitable for an academic final-year project. It avoids unnecessary complexity while still delivering good performance. The design focuses on clarity and reproducibility rather than high-throughput production-level caching.

In future enhancements, if the project is scaled to handle much larger datasets or real-time data streams, more advanced caching strategies such as in-memory databases or distributed caching can be introduced.

7. Non-Functional Requirements

7.1. Security Aspects: Since the project is a local academic pipeline running entirely on a single machine inside a Jupyter Notebook, security requirements are kept minimal and straightforward. There is no user authentication, login system, or external access involved. The dataset (`energy_consumption_levels.csv`) contains only non-sensitive energy consumption data with no personally identifiable information. The SQLite database file (`energy_demand.db`) and all notebook files are stored locally on the user's machine. File-level protection provided by the operating system is sufficient for this project. No encryption or advanced security features were implemented as they are not relevant to the current scope.

7.2. Performance Aspects: The complete ETL pipeline executes in just a few seconds on a standard laptop. All operations are performed efficiently in-memory using highly optimized Pandas functions such as `GroupBy` and `resample`. The SQLite database provides fast read and write performance for reporting queries. Because the dataset contains only 8784 rows, there is no need for distributed computing or complex performance tuning. The design ensures quick execution, easy reproducibility, and smooth demonstration during project evaluation. The system is scalable in future — the same logic can be applied to much larger datasets by switching to tools like Spark or cloud-based solutions.

Overall, the non-functional requirements focus on simplicity, maintainability, and educational value rather than production-level scalability or high-security features.

8. References

1. Pandas Development Team. (2026). *Pandas Documentation*. Retrieved from <https://pandas.pydata.org/docs/>
2. SQLite Development Team. (2026). *SQLite Documentation*. Retrieved from <https://www.sqlite.org/docs.html>
3. Mendeley Data Repository. (2018). *A 12-month Data of Hourly Energy Consumption Levels (2016)*. Retrieved from Mendeley Data.
4. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.
5. McKinney, W. (2012). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O'Reilly Media.
6. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 51(1), 107–113.
7. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley.
8. Yenepoya (Deemed to be University). (2026). BCA Final Year Project Guidelines – Big Data Analytics.
9. Seaborn Development Team. (2023). Seaborn Statistical Data Visualization. Retrieved from <https://seaborn.pydata.org/>